

TECHNICAL HANDOFF

CARLA Nav2 AVL

Perception, costmap, Navigation2, and operator startup guide for the AVL IGVC ground robot

Document	Value
Repository	arassal/carla-nav2-avl
Reviewed branch	main
Primary source snapshot	carla-nav2-avl 2684f876d503ce222289d5a3f2bb824492ea9fda (2684f87), committed July 29, 2026
Vehicle source snapshot	IGVC_ROS2 d40e18aa480f76673b46a379ab7cb4618f279d75 (d40e18a), committed June 1, 2026
Review date	August 5, 2026
Intended recipient	Developer or technical lead assuming ownership
Document status	Expanded draft for review - includes operator runbook; not an operational sign-off

Purpose and limitation

This document translates the repository into an ownership handoff. It distinguishes active implementation from legacy/stub code and flags documentation drift. It does not assert the current physical state of the robot, team availability, credentials, or safety approval. Those items require confirmation by the project owner.

0. Contents and first-day path

Section	What it answers
1	Executive summary
2	Scope and system boundaries
3	Architecture and runtime data flow
4	Repository map and ownership
5	Current state and evidence
6	Environment and dependencies
7	Build, launch, and operating runbook
8	Configuration that matters
9	Testing and acceptance
10	Safety, troubleshooting, and recovery
11	Risks, gaps, and documentation drift
12	Handoff checklist
A	Operator connection and startup guide
B	Field command card
13	Source references

Recommended first-day sequence

- Read sections 1, 4, 5, and 11 before touching code or the robot.
- Confirm the current vehicle state with the project lead and inspect the deployed commit on dinosaur.
- Run the offline perception tests in Ubuntu 22.04 / ROS 2 Humble, then build only perception_costmap.
- Bring up sensors and perception with Nav2 disarmed. Verify topics, TF, and visual outputs before any motion.
- Only arm Navigation2 with a human on the E-stop and a documented field-test plan.

Highest-value fact

The maintained core is perception_costmap plus driving_seg. Most other ROS packages and the root scripts directory are legacy or stubs. Start from the maintained package, not from the apparent breadth of the repository.

1. Executive summary

The project provides a ROS 2 perception-to-navigation pipeline for a slow skid-steer IGVC robot operating on grass between painted white lines and traffic cones. CARLA supports simulation work on x86, while the physical vehicle ('dinosaur') uses an NVIDIA Jetson AGX Orin with three ZED X cameras, a VLP-16 lidar, an Xsens IMU/GNSS, and a Teensy/SparkMAX drive interface. CARLA does not run on the Jetson; the intended portability boundary is the sensor source and actuator sink.

SENSOR SOURCES	PERCEPTION CORE	NAVIGATION + CONTROL
3x ZED X RGB cameras VLP-16 lidar (currently disabled in active config) TF and EKF from external avros bring-up	Segmentation + object detection Per-camera IPM into fused BEV Temporal filter + cost shaping /perception/costmap	costmap_to_cloud bridge Nav2 ObstacleLayer + inflation Planner + regulated pure pursuit /cmd_vel (isolated by default)

Operationally, the active real-car path is camera-led: the current vehicle configuration enables HSV road segmentation, YOLO obstacles, cones, and white-line handling, but sets `use_lidar: false`. The perception grid is converted to a PointCloud2 stream for Nav2's ObstacleLayer. Navigation output is remapped away from the actuator topic unless `NAV2_ARMED=1` is explicitly set.

Status at handoff

Area	Assessment	Evidence / implication
Perception core	Implemented and documented as maintained	ROS-free geometry/segmentation/occupancy modules, ROS node, configs, tools, and offline tests are present.
Real-car operations	Substantial operational glue exists	Boot service, tmux bring-up, camera recovery, dashboards, and Nav2 launch scripts are checked in; many are host-specific.
Performance	Acceptance not yet demonstrated in current snapshot	Last recorded TwinLite CUDA run was about 4-5 Hz against an 8 Hz target; current default changed to HSV and needs a fresh benchmark.
Geometry	Values exist; field truth still needs confirmation	Current dinosaur YAML includes camera mount/calibration values dated July 9, while multiple docs still call calibration open.
Navigation safety	Safe default, field validation required	Nav2 is disarmed by default; road keeping depends on a cost threshold contract between perception and cloud conversion.
Documentation	Useful but internally inconsistent	Older design/status documents describe obsolete StaticLayer and phase gaps. The active config and launch files should win.

2. Scope and system boundaries

In scope

- Three-camera road, white-line, person/vehicle, and cone perception.
- Bird's-eye-view fusion into a robot-centric OccupancyGrid.
- Cost shaping, temporal filtering, and per-class safety halos.
- Conversion of the perception grid to obstacle points consumed by Navigation2.
- Local/global rolling Nav2 costmaps, path planning, and regulated pure pursuit control.
- CARLA feed/test helpers, Jetson deployment guidance, boot automation, visualization, and recovery scripts.
- Standalone multi-model driving_seg research/tooling for richer semantic highlighting and course-class training.

External or intentionally excluded

Boundary	Owner / source	Handoff consequence
Physical sensor drivers, TF, and EKF	Separate IGVC_ROS2 / avros bring-up repository	This repository cannot produce live data without that stack. Confirm compatible branch and topic names.
Physical motor actuation	Existing actuator/CAN stack on the robot	Do not run duplicate actuator nodes. Navigation output is intentionally isolated until armed.
TensorRT engines	Built on the target Jetson	The repository ships model weights but engines are GPU/CUDA/TensorRT-specific and must be regenerated.
Credentials and network access	Project owner	No passwords or secrets belong in this handoff. Obtain robot SSH/Tailscale access separately.
Competition safety approval	Team safety lead	Software presence is not authorization to drive. E-stop and supervised-test procedures must be confirmed.

Authoritative-source order

- For runtime behavior: active launch files and active YAML configuration.
- For algorithm behavior: perception_costmap source and its tests.
- For actual deployment: inspect the files and commit running on dinosaur, then compare with main.
- For intent/history: README, DESIGN.md, DEPLOY.md, REPRODUCE.md, and FINALIZE.md, with dates considered.

3. Architecture and runtime data flow

Perception flow

- Each configured camera subscribes to rectified RGB and CameraInfo using sensor-data QoS.
- A selectable segmenter produces the road mask. The active dinosaur config defaults to HSV; TwinLiteNet is available but requires external model code and a valid engine.
- YOLO produces class-aware camera obstacle masks. Cone detection is separately configured; white-line pixels are treated as non-drivable boundaries.
- Each camera mask is projected through inverse perspective mapping into a shared robot-centric bird's-eye-view grid. Camera contributions are fused.
- A temporal filter suppresses obstacle flicker. Occupancy shaping assigns free, off-road, unknown, lethal, and inflated per-class costs.
- The node publishes /perception/costmap, /perception/known, and /perception/obstacle_points.

Active Nav2 flow

The current `real_nav2_params.yaml` deliberately omits `StaticLayer`. A bridge raycasts the robot-relative perception grid into `/perception/costmap_cloud`. Both rolling costmaps consume this cloud through `ObstacleLayer`, then apply `InflationLayer`. `Navfn` produces the path and `RegulatedPurePursuitController` follows it.

Do not revive the obsolete `StaticLayer` design casually
Older `DESIGN.md` text describes feeding `/perception/costmap` to `StaticLayer`. The active Nav2 config explains why that approach was abandoned: a rotating robot-relative grid cannot be represented correctly by the axis-aligned `StaticLayer`. Treat `real_nav2_params.yaml` and `real_nav2_launch.py` as the current design.

Key topics and frames

Interface	Type / frame	Direction and role
/zed_front/.../image and camera_info	sensor_msgs/Image, CameraInfo	Input; analogous left/right topics are defined in dinosaur YAML.
/velodyne_points	sensor_msgs/PointCloud2	Configured lidar input, but ignored while use_lidar is false.
/perception/costmap	nav_msgs/OccupancyGrid; base_link	Primary perception result, 20 m x 20 m at 0.1 m/cell in the vehicle config.
/perception/costmap_cloud	sensor_msgs/PointCloud2	Bridge output and active Nav2 obstacle source.
/odometry/filtered	nav_msgs/Odometry; odom/base_link	Vehicle localization used by run helpers and Nav2.
/nav2/cmd_vel	geometry_msgs/Twist	Default disarmed Nav2 output.
/cmd_vel	geometry_msgs/Twist	Real actuator command only when explicitly armed.
map -> odom -> base_link -> sensors	TF tree	Perception publishes in base_link; active Nav2 global frame is odom because no map anchor exists.

4. Repository map and ownership

Path	Status	Use
ros2_ws/src/perception_costmap/	Maintained core	Perception node, ROS-free algorithm modules, launch, configs, tests, deployment tools.
driving_seg/	Active research/tooling	Multi-model semantic highlighting and course-class training for cones and white lines.
models/	Required artifacts	Committed YOLO and TwinLite weights. Target-specific TensorRT engines are not committed.
ros2_ws/src/carla_msgs/	Simulation support	CARLA vehicle-control message definition.
ros2_ws/src/world_setup/	Legacy/stub	CARLA bridge/world setup; not part of the maintained real-car path.
collision_guard, controller, route_planner, sdc_bringup, sdc_common	Legacy/stub	Do not extend without explicit review. Package presence does not imply readiness.
scripts/	Legacy CARLA helpers	Historical x86 simulation launch and validation scripts; not the primary real-car runbook.
perception/	Prototype	Original prototype scripts retained for attribution/history.
docs/plans, FINALIZE.md	Planning/history	Useful intent and acceptance ideas; may lag implementation.

Core perception modules

Module	Responsibility
costmap_node.py	ROS wiring, parameters, subscriptions, staleness handling, multi-camera fusion, publishers.
segmentation.py	HSV and TwinLite segmentation backends, including TensorRT adapter paths.
obstacles.py	YOLO class-aware obstacle masks, cone detector, and lidar filtering helpers.
bev.py	IPM/homography math and camera-to-grid projection.
temporal.py	Per-cell confidence filter for transient obstacles.
occupancy.py	Cost-array construction, off-road shaping, unknown infill, and per-class danger zones.
deploy/costmap_to_cloud.py	Current perception-to-Nav2 bridge.
deploy/real_nav2_launch.py	Starts bridge and Nav2 servers; applies the safe command-topic remap.

Team metadata in repository

The repository lists Alexander (@arassal) as project lead, with jchy05, AdamCastillo07, and Adrian (@Ad-Tap) as contributors/mentees. Confirm current roles, availability, and operational ownership before treating these as escalation contacts.

5. Current state and evidence

Snapshot

Item	Observed in reviewed main snapshot
Commit	2684f876d503ce222289d5a3f2bb824492ea9fda - Add .mailmap: coalesce author identities
Commit timestamp	July 29, 2026 (repository metadata)
Required vehicle repository	IGVC_ROS2 at d40e18aa480f76673b46a379ab7cb4618f279d75 (d40e18a), committed June 1, 2026
Maintained package version	perception_costmap 0.1.0
Documented offline result	39 perception tests reported green in repository docs
Independent test run for this handoff	Not run: the local Windows documentation environment lacked pytest/ROS dependencies
Vehicle config	3 ZED X cameras; HSV segmentation; front-camera YOLO; cones and white lines enabled; lidar disabled
Grid	x -4..16 m, y -10..10 m, 0.1 m resolution, base_link, 10 Hz requested publish rate
Nav2 state	Odom-relative rolling local/global costmaps; cloud bridge; disarmed by default

What appears complete in source

- Multi-camera BEV fusion and robot-centric grid construction.
- HSV and TwinLite segmentation interfaces, YOLO detection, cone detection, and white-line handling.
- Per-class danger zones for people, vehicles, cones, and generic obstacles.
- Active Nav2 bridge/configuration with an explicit safe command remap.
- Car deployment scripts, boot service, visualization, dashboard, and GMSL camera recovery procedures.
- Tools for IPM overlays, benchmarking, TensorRT export, shadow robustness, and route/goal validation.

What is not proven by the repository alone

- That the physical robot is currently running commit 2684f87 or that the boot service is healthy today.
- That the current configuration sustains at least 8 Hz with all enabled perception stages.
- That cone/line geometry is within 0.1 m in the competition field.
- That Navigation2 avoids cones and people during repeatable supervised live runs.
- That lidar is ready for production fusion; it is explicitly disabled in the reviewed vehicle config.
- That the external sensor/actuator repositories and topic contracts still match this snapshot.

6. Environment and dependencies

Component	Documented target
Vehicle compute	NVIDIA Jetson AGX Orin 64 GB ('dinosaur')
OS / middleware	JetPack 6.1 / L4T R36.4.x, Ubuntu 22.04, ROS 2 Humble

Component	Documented target
GPU stack	CUDA 12.6 and TensorRT 10.3 recorded in deployment notes
Cameras	3x Stereolabs ZED X over GMSL: front, left, right
Lidar	Velodyne VLP-16
Localization	Xsens MTi-680G IMU/GNSS plus external EKF/TF bring-up
Drive	Teensy 4.1 to dual REV SparkMAX over CAN; differential/skid-steer
Python ML	Jetson-specific torch 2.8.0, torchvision 0.23.0, numpy<2, ultralytics without replacing NVIDIA torch
ROS middleware	rmw_cyclonedds_cpp with the avros CycloneDDS configuration on the vehicle

Jetson dependency trap

Do not install generic PyPI torch on the Jetson. The repository records that this can produce a CUDA-incompatible wheel. Use the Jetson AI Lab index for JetPack 6 / CUDA 12.6, keep torch/torchvision at the documented compatible versions, and pin numpy below 2 to protect the ROS 2 Humble cv_bridge chain.

Required external components

- ROS 2 Humble, Navigation2, ZED SDK/wrappers, VLP-16 driver, Xsens/robot_state_publisher/EKF bring-up.
- The separate IGVC_ROS2 repository for sensor bring-up and vehicle integration.
- TwinLiteNetPlus source only if using the TwinLite backend.
- Target-built TensorRT engines for yolov8n.pt and cone_det.pt; optionally TwinLite once its exporter path is validated.
- web_video_server, tmux, and systemd support for the as-deployed operations scripts.

7. Build, launch, and operating runbook

Portable checkout verification

```
git clone https://github.com/arassal/carla-nav2-avl.git
cd carla-nav2-avl/ros2_ws/src/perception_costmap
PYTHONPATH=. python3 -m pytest test -q
```

Build only the maintained ROS package

```
cd carla-nav2-avl/ros2_ws
colcon build --packages-select perception_costmap
source install/setup.bash
```

Prepare target-specific models

```
export AVL_MODELS_DIR="$(git rev-parse --show-toplevel)/models"
cd src/perception_costmap
python3 tools/export_trt.py --weights "$AVL_MODELS_DIR/yolov8n.pt"
python3 tools/export_trt.py --weights "$AVL_MODELS_DIR/cone_det.pt"
```

Vehicle launch path

Bring up external sensors/TF and EKF, start the three ZED wrappers sequentially, launch perception and visualization, then start Nav2 disarmed. Verify each gate before proceeding. The exact connection, service, recovery, disarmed-planning, and arming commands are consolidated in Appendix A; the field command card is Appendix B.

8. Configuration that matters

Setting / contract	Reviewed value	Why it matters
segmentation_method	hsv	Only turn-key path from shipped artifacts; TwinLite requires external source/engine.

Setting / contract	Reviewed value	Why it matters
obstacle_method	yolo	Camera obstacle detection is learned-model based.
yolo_cameras	front	Only the front camera runs general YOLO in the current vehicle config.
use_cones / use_white_lines	true / true	Course-specific perception is enabled in configuration and code.
use_lidar	false	Raw VLP-16 does not currently contribute to the active perception/Nav2 obstacle path.
publish_rate	10 Hz	Requested rate, not proof of achieved throughput.
grid	20 x 20 m; 0.1 m/cell	Vehicle-centered perception region: x -4..16, y -10..10.
offroad_cost	97	Must remain synchronized with costmap_to_cloud obstacle threshold or road-keeping silently disappears.
unknown_cost	25	Blind areas are mildly penalized rather than left as ROS unknown.
image_stale_sec	1.0	Raised to prevent flicker from the observed front-camera rate and scheduling jitter.
Nav2 global frame	odom	No map/SLAM/GPS anchor is configured; navigation is odom-relative.
Nav2 safety gate	NAV2_ARMED != 1	Default remaps commands to /nav2/cmd_vel so motors cannot receive them.

Camera mount values in active vehicle YAML

Camera	Position (x, y, z m)	Pitch / yaw	Topic prefix
front	0.6795, 0.000, 0.4476	17 / 0 deg	/zed_front/zed_node/...
left	0.098, 0.286, 0.6126	18 / 94 deg	/zed_left/zed_node/...
right	0.098, -0.286, 0.6126	20 / -86 deg	/zed_right/zed_node/...

Configuration-change rule

Treat cost thresholds, frames, topic names, model paths, camera geometry, and arming behavior as interface contracts. Change them with a test or measured field observation, update the runbook, and verify from a fresh clone. Avoid hardcoded home paths outside deploy scripts.

9. Testing and acceptance

Repository-reported tests

The project documentation reports 39 offline perception tests. Test files cover CARLA conversion, cheirality/orientation, course classes, detection, footprint geometry, segmentation, temporal filtering, and utilities. This handoff build did not execute them because the local Windows documentation environment did not have the Ubuntu/ROS Python test stack installed.

Minimum takeover verification

Level	Check	Pass condition
Offline	pytest in perception_costmap	All documented tests pass from a clean Ubuntu checkout.
Build	colcon build --packages-select perception_costmap	Clean build with no package-resolution errors.
Model	Export and benchmark YOLO/cone engines on target	Engines load without fallback; timing is recorded per stage.
Topics	Sensor and output topic rates	No stale streams; /perception/costmap sustained at least 8 Hz target.
Geometry	Asymmetric left/right fixtures plus tape-measured objects	No mirror errors; each camera places objects within 0.1 m / one cell target.
Perception	Person, cone, and white-line test	Lethal marking within 300 ms; person clears within 500 ms; course boundaries remain closed.
Planning	Plan-only obstacle corridor	Path avoids detected obstacle with required clearance; no motor command on /cmd_vel.
Motion	Supervised low-speed field test	E-stop proven; one actuator; speed caps respected; no boundary crossing/contact.
Reliability	Cold boot and three repeated runs	Hands-off service recovery and consistent completion without silent failure.

Evidence to archive with each release

- Exact Git commit, vehicle config diff, external bring-up commit, model hashes, and TensorRT build versions.
- pytest and colcon logs, topic-rate captures, per-stage benchmark output, and TF diagnostics.
- RViz/dashboard screenshots or a rosbag showing cone, person, and white-line behavior.
- Field-test date, operator, safety observer, E-stop check, weather/lighting, and failures observed.

10. Safety, troubleshooting, and recovery

Motion safety

Never set NAV2_ARMED=1 merely to see whether the stack works. First verify TF, odometry, perception, planning, costmap thresholds, command topic isolation, and E-stop function. Keep a human on the E-stop for every initial or changed configuration run.

The detailed symptom-to-response table and exact recovery commands are consolidated in Appendix A4. The non-negotiable rules are: start TF before ZED wrappers, recover cameras through `clean_camera_restart.sh`, require /perception/costmap_cloud before trusting Nav2, keep offroad_cost synchronized with the bridge threshold, verify model loading, and allow exactly one actuator process.

11. Risks, gaps, and documentation drift

Priority	Risk / discrepancy	Handoff action
P0	Physical system state is not knowable from the repository.	Confirm deployed commit, service status, sensor health, E-stop, and current operator notes before any motion.
P0	Nav2 can drive the real actuator when NAV2_ARMED=1.	Require an explicit test authorization and human E-stop coverage; preserve disarmed default.
P1	Active vehicle config disables lidar.	Decide whether camera-only avoidance is acceptable. If enabling lidar, add and validate the complete perception/Nav2 observation path.

Priority	Risk / discrepancy	Handoff action
P1	Performance evidence is stale relative to current config.	Benchmark current HSV + front YOLO + cone/line stack on dinosaur and record sustained topic rate.
P1	Calibration status conflicts across files.	Repeat tape-measure/asymmetric validation and update README/DESIGN/DEPLOY consistently.
P1	FINALIZE.md says cones/white lines are not integrated, while source/config show integration.	Treat source as newer, test behavior, then retire or update stale phase text.
P1	DESIGN.md describes StaticLayer, while active Nav2 uses costmap_to_cloud.	Update architecture documentation and remove ambiguity around deprecated nav2_costmap_params.yaml.
P1	External avros/IGVC repository is a runtime prerequisite.	Pin and document the compatible commit and topic/TF contract.
P2	Host-specific deploy scripts encode /home/dinosaur and network assumptions.	Keep them as the as-run path but add a parameterized installation procedure or environment file.
P2	TwinLite is documented as faster-quality work but is not turn-key.	Either productize its exporter/runtime or explicitly keep HSV as the supported baseline.
P2	Legacy/stub packages increase onboarding confusion.	Mark them clearly in package metadata or move them under a legacy area after team agreement.

Recommendations and decisions for the next team

- Supported production perception backend: HSV versus TwinLite/TensorRT.
- Camera-only versus camera-plus-lidar safety architecture.
- Canonical navigation architecture and deprecation plan for obsolete configs/tools.
- Field-calibration owner, procedure, acceptance tolerance, and recertification trigger.
- Release evidence, deployment owner, rollback method, and safety sign-off authority.

12. Handoff checklist

Ownership and access

- [] Confirm project lead, perception owner, navigation owner, vehicle operator, and safety approver.
- [] Obtain repository, robot SSH/Tailscale, dashboard, and lab access through approved channels.
- [] Record the deployed commit for this repo and the external IGVC_ROS2/avros stack.
- [] Inventory robot services and confirm only one actuator node can run.

Technical and operational gates are not repeated here: use the acceptance matrix in section 9 and the executable checklists in Appendix A. Record every result against the exact deployed commits and model hashes.

Suggested 30-day takeover plan

Window	Outcome
Days 1-3	Reproduce tests/build, map deployed versions, meet owners, and verify robot access without motion.
Week 1	Recreate target engines, validate topics/TF/config, benchmark current perception, and resolve documentation drift.
Week 2	Complete geometry and obstacle acceptance tests; decide the lidar and segmentation production baseline.
Weeks 3-4	Run supervised field acceptance, harden boot/recovery, archive evidence, and publish an internally approved operating runbook.

A. Operator connection and startup guide

Scope of this guide

This procedure explains the operating path encoded in the two repositories. It does not confirm that the vehicle is currently powered, reachable, mechanically safe, or running the reviewed commits. Team authorization, credentials, a trained operator, and a tested physical E-stop are required before motion.

Operator-computer software

Tool	Purpose	Operator action
Tailscale	Private network path to the Jetson	Join the team-approved tailnet. Obtain access from the administrator; do not create a parallel network.
OpenSSH	Command-line control	Use Windows Terminal/PowerShell, macOS Terminal, or Linux shell. PuTTY is optional.
Web browser	Joystick/E-stop and perception dashboard	Use a current browser. Confirm the Jetson identity before accepting its self-signed joystick certificate.
NoMachine Client	Jetson desktop and RViz	Connect to the Jetson graphical session. RViz waits for this display and uses software rendering.

Repository-recorded connection

Item	Recorded value	Caution
Tailscale address	100.93.121.3	Snapshot value; confirm in the tailnet before use.
Linux user	dinosaur	SSH credentials or keys must come from the team.
SSH alias	jetson	Only works if the operator's ~/.ssh/config defines it.
Vehicle workspace	/home/dinosaur/IGVC	The older /home/dinosaur/AVROS directory is documented as obsolete.
Perception workspace	/home/dinosaur/carla-nav2-avl	Deployment scripts assume this exact path.

Connect

```
# From the operator computer
tailscale ping 100.93.121.3
ssh dinosaur@100.93.121.3

# If the team-installed SSH alias exists
ssh jetson
```

Connection failure

If SSH fails, confirm Jetson power, Tailscale status, the current address, and SSH-key authorization. Do not guess passwords repeatedly or bypass the approved network.

A1. Physical pre-start and boot

For the first startup after a handoff, place the vehicle on rated stands or use an approved clear test area. The operator must be able to stop both tracks immediately.

Physical checklist

- [] Physical E-stop identified, reachable, and function-tested by the team operator.
- [] Motors disabled or tracks safely clear of the ground for software checkout.
- [] Battery condition and polarity verified; no damaged or loose power wiring.

- [] Jetson DC connector secured. Repository notes record previous hard power cuts from this connector.
- [] Teensy USB/serial cable secured; SparkMAX controllers and CAN wiring inspected.
- [] ZED Link/GMSL connections secured; camera lenses clean and unobstructed.
- [] Velodyne power and Ethernet path inspected. A July 16 note reported eno1 NO-CARRIER.
- [] Clear exclusion area established around the tracks; safety observer assigned.

Expected boot services

Service	Starts	Important ownership
avros-webui	actuator_node, webui_node, HTTPS port 8000	Owns the Teensy serial interface and accepts web joystick or /cmd_vel. Exactly one actuator_node.
percept-stack	Sensors, EKF, three ZED wrappers, perception, visualization, ports 8080/8090, RViz waiter	Uses the dedicated tmux socket named percept. Three cameras start about 40 seconds apart.

Check service state

```
sudo systemctl status avros-webui --no-pager
sudo systemctl status percept-stack --no-pager
tmux -L percept list-windows -t percept
```

Expected tmux windows include sensors, ekf, zed_front, zed_left, zed_right, costmap, viz, wvs, www, costmap_rgb, and rviz. A complete camera startup may take several minutes.

A2. Operator interfaces and live monitoring

Interface	Address / command	Purpose
Perception dashboard	http://100.93.121.3:8090	Fused BEV, rendered costmap, and three camera streams.
Joystick and software E-stop	https://100.93.121.3:8000	Direct actuator control through avros_webui. Self-signed certificate is expected.
MJPEG backend	http://100.93.121.3:8080	Stream server used by the dashboard; normally not operated directly.
RViz desktop	NoMachine -> 100.93.121.3	Graphical ROS inspection. The checked-in launcher waits for a NoMachine display.
Process console	tmux -L percept attach -t percept	Inspect the active sensor, camera, perception, and visualization windows.

Useful tmux controls

Keys	Effect
Ctrl-b, n	Next window
Ctrl-b, p	Previous window
Ctrl-b, w	List windows
Ctrl-b, d	Detach without stopping the stack

Log inspection

```
ls -l /tmp/sensors.log /tmp/ekf.log /tmp/zed_*.log /tmp/costmap.log
tail -n 100 /tmp/costmap.log
grep -Ei "engine|model|weights|fallback|error|warning" /tmp/costmap.log
```

Model-path check
 perception_dinosaur.yaml uses AVL_MODELS_DIR, but the reviewed boot script does not visibly export it. Confirm that the service environment provides the variable and that /tmp/costmap.log reports successful model/engine loading rather than a silent fallback.

A3. ROS environment and health gates

Prepare every diagnostic shell

```
source /opt/ros/humble/setup.bash
source /home/dinosaur/IGVC/install/setup.bash
source /home/dinosaur/carla-nav2-avl/ros2_ws/install/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export
CYCLONEDDS_URI=file:///home/dinosaur/IGVC/install/avros_bringup/share/avros_bringup/config/cyclonedds.xml
export AVL_MODELS_DIR=/home/dinosaur/carla-nav2-avl/models
```

Required checks before Nav2

Gate	Command	Required observation
Localization	ros2 topic hz /imu/data ros2 topic hz /odometry/filtered	Stable rates; no unexplained gaps or stationary rotation.
TF	ros2 run tf2_ros tf2_echo odom base_link	Continuous, coherent transform.
Front/left/right RGB	ros2 topic hz /zed_front/zed_node/rgb/color/rect/image ros2 topic hz /zed_left/zed_node/rgb/color/rect/image ros2 topic hz /zed_right/zed_node/rgb/color/rect/image	All three publish current frames; topic existence alone is insufficient.
Perception	ros2 topic hz /perception/costmap	Sustained rate measured; project target is at least 8 Hz.
Perception geometry	Dashboard or RViz	Forward/left/right orientation correct; cone, person, and white line appear in the correct grid location.
Actuator	pgrep -af actuator_node ros2 topic hz /avros/actuator_state	Exactly one actuator process; healthy state publishing.
Serial owner	sudo lsusb /dev/serial/by-id/usb-Teensyduino_USB_Serial_18639150-if00	Exactly one expected holder.
Lidar physical link	ip link show eno1 ethtool eno1	Link detected: yes, if lidar is expected.
Lidar topic	ros2 topic hz /velodyne_points	Live points. Note that active perception config still sets use_lidar: false.

Visual acceptance

- A left-side obstacle must render left, and a right-side obstacle must render right.
- Cones and people must become lethal; painted white lines must close the course boundary.
- Removed obstacles must clear within the accepted time; stale frames must not remain trusted.
- Blind regions must not appear as confidently free because of a visualization or threshold error.

A4. Manual restart and camera recovery

Stationary-only operation
 The restart procedures stop and replace live perception processes. Run them only while the vehicle is stationary and autonomous motion is disabled.

Full perception restart

```
bash /home/dinosaur/carla-nav2-avl/ros2_ws/src/perception_costmap/deploy/full_stack_restart.sh
```

The script kills the existing percept tmux session, stops old ZED/perception/stream processes, starts sensors and EKF, waits, launches the three cameras sequentially, then starts perception and visualization. Manual mode places tmux in a systemd user scope so it survives SSH teardown.

Verify completion

```
tmux -L percept list-windows -t percept
tail -n 100 /tmp/costmap.log
ros2 topic hz /perception/costmap
```

Camera recovery

Use this for CAMERA STREAM FAILED TO START, Argus timeouts, blank video, or a stopped camera topic:

```
bash /home/dinosaur/carla-nav2-avl/ros2_ws/src/perception_costmap/deploy/clean_camera_restart.sh
```

The recovery script stops all wrappers, restarts nvargus-daemon and zed_x_daemon, then starts the cameras about 40 seconds apart. Do not restart zed_x_daemon under live wrappers by hand; the repository records that this can wedge the remaining streams.

Common symptom map

Symptom	First check
ZED waits for static transforms	Confirm robot_state_publisher and the external sensor/TF stack started first.
Dashboard loads but images are blank	Check web_video_server, image topic rates, and the camera logs.
Perception grid is empty	Check current camera frames, model fallback messages, and config topic names.
Known grid flickers	Check camera rates/timestamps and image_stale_sec; current vehicle config uses 1.0 s.
Nav2 sees no obstacles	Confirm costmap_to_cloud is running and /perception/costmap_cloud publishes.

A5. Safe Navigation2 activation

Start disarmed

```
cd /home/dinosaur/carla-nav2-avl/ros2_ws
unset NAV2_ARMED
ros2 launch src/perception_costmap/deploy/real_nav2_launch.py
```

The disarmed launch remaps controller output to /nav2/cmd_vel. The actuator listens to the real /cmd_vel, so this mode permits planning and controller inspection without intentional motor commands.

Verify Nav2

```
ros2 lifecycle get /controller_server
ros2 lifecycle get /planner_server
ros2 lifecycle get /bt_navigator
ros2 topic hz /perception/costmap_cloud
ros2 topic info /nav2/cmd_vel -v
ros2 topic info /cmd_vel -v
```

All lifecycle nodes must be active. The perception cloud must publish. Use RViz to send a short odom-relative goal and confirm the planned path clears a test obstacle while Nav2 remains isolated from /cmd_vel.

Pre-arm gate

- [] Approved clear area, trained operator, safety observer, and tested physical E-stop.
- [] Browser E-stop open and verified; only one actuator node and serial owner.
- [] Stable TF/odometry, three current camera streams, acceptable costmap rate, and correct geometry.

- [] Cone/person/line behavior verified in both perception and the Nav2 costmap.
- [] Disarmed plan avoids the obstacle; speed and goal distance approved.
- [] Lidar-disabled/camera-only limitation explicitly accepted for this test.
- [] Independent distance watchdog running in a separate shell.

Distance watchdog example

```
cd /home/dinosaur/carla-nav2-avl/ros2_ws/src/perception_costmap
python3 deploy/distance_watchdog.py --limit 2.0
```

A6. Arming, first goal, E-stop, and shutdown

Motion boundary

The following commands can cause the physical tracks to move. They require the team's operating authority, a human on the physical E-stop, and a cleared test area.

Arm Nav2

Stop the disarmed Nav2 launch with Ctrl-C. NAV2_ARMED is read when the launch file starts, so it must be set before relaunch:

```
cd /home/dinosaur/carla-nav2-avl/ros2_ws
export NAV2_ARMED=1
ros2 launch src/perception_costmap/deploy/real_nav2_launch.py
```

Verify /cmd_vel publisher/subscriber ownership immediately. Do not launch the separate avros_bringup navigation stack at the same time, and do not start a second actuator node.

First short goal

The goal helper's default is about 6.1 m, which is inappropriate for a first handoff test. Pass a deliberately short approved distance, for example:

```
cd /home/dinosaur/carla-nav2-avl/ros2_ws/src/perception_costmap
python3 deploy/send_goal_ahed.py 1.0
```

Watch the physical robot, RViz, browser E-stop, /cmd_vel, /odometry/filtered, watchdog, and actuator logs simultaneously.

Emergency stop

- Primary: use the physical E-stop.
- Secondary: use the browser E-stop at <https://100.93.121.3:8000>.
- Command-line backup: publish a one-shot ActuatorCommand with estop: true.

```
ros2 topic pub --once /avros/actuator_command avros_msgs/msg/ActuatorCommand '{estop: true}'
```

Normal shutdown

- Stop Nav2 with Ctrl-C and unset NAV2_ARMED.
- Confirm the robot is stopped and the motor system is disabled according to the team procedure.
- Stop perception with sudo systemctl stop percept-stack if ending the session.
- Stop joystick/actuator service with sudo systemctl stop avros-webui only after safe motor stop.
- Shut down Linux with sudo poweroff; wait for shutdown before removing power.

B. Field command card

Use condition

This page is a reminder for a trained operator, not a substitute for Appendix A. Start on stands or in an approved clear area. Keep Nav2 disarmed until every gate passes.

All ROS commands below assume the shell preparation in Appendix A3 has already been completed.

Task	Command / address
SSH	<code>ssh dinosaur@100.93.121.3</code>
Service health	<code>sudo systemctl status avros-webui percept-stack --no-pager</code>
Perception console	<code>tmux -L percept attach -t percept</code>
Perception dashboard	<code>http://100.93.121.3:8090</code>
Joystick / software E-stop	<code>https://100.93.121.3:8000</code>
NoMachine / RViz	Connect to 100.93.121.3 as dinosaur
Perception log	<code>tail -n 100 /tmp/costmap.log</code>
Full perception restart	<code>bash /home/dinosaur/carla-nav2-avl/ros2_ws/src/perception_costmap/deploy/full_stack_restart.sh</code>
Camera recovery	<code>bash /home/dinosaur/carla-nav2-avl/ros2_ws/src/perception_costmap/deploy/clean_camera_restart.sh</code>
Costmap rate	<code>ros2 topic hz /perception/costmap</code>
Disarmed Nav2	<code>cd /home/dinosaur/carla-nav2-avl/ros2_ws; unset NAV2_ARMED; ros2 launch src/perception_costmap/deploy/real_nav2_launch.py</code>
Distance watchdog	<code>cd /home/dinosaur/carla-nav2-avl/ros2_ws/src/perception_costmap; python3 deploy/distance_watchdog.py --limit 2.0</code>
Arm Nav2	<code>cd /home/dinosaur/carla-nav2-avl/ros2_ws; export NAV2_ARMED=1; ros2 launch src/perception_costmap/deploy/real_nav2_launch.py</code>
Short goal	<code>cd /home/dinosaur/carla-nav2-avl/ros2_ws/src/perception_costmap; python3 deploy/send_goal_ahead.py 1.0</code>
CLI E-stop backup	<code>ros2 topic pub --once /avros/actuator_command avros_msgs/msg/ActuatorCommand '{estop: true}'</code>
Return to safe default	<code>Stop Nav2; unset NAV2_ARMED</code>
Stop services	<code>sudo systemctl stop percept-stack avros-webui</code>
Power off Jetson	<code>sudo poweroff</code>

Do not proceed if

- No physical E-stop operator; more than one actuator node; serial-port collision.
- Missing/stale camera, unstable TF or odometry, incorrect left/right geometry, empty Nav2 obstacle cloud.
- Unexplained model fallback, costmap below accepted rate, lidar assumed active without evidence.
- The physical robot or deployed software differs materially from this reviewed snapshot.

13. Source references

This handoff was produced from read-only local clones of the two public GitHub repositories. No repository files, branches, issues, pull requests, or remote state were changed.

Source	Location
Repository	https://github.com/arassal/carla-nav2-avl
Root orientation	https://github.com/arassal/carla-nav2-avl/blob/main/README.md
Reproduction path	https://github.com/arassal/carla-nav2-avl/blob/main/REPRODUCE.md
Architecture	https://github.com/arassal/carla-nav2-avl/blob/main/ros2_ws/src/perception_costmap/DESIGN.md
Package guide	https://github.com/arassal/carla-nav2-avl/blob/main/ros2_ws/src/perception_costmap/README.md
Jetson deployment	https://github.com/arassal/carla-nav2-avl/blob/main/ros2_ws/src/perception_costmap/DEPLOY.md
Real vehicle config	https://github.com/arassal/carla-nav2-avl/blob/main/ros2_ws/src/perception_costmap/config/perception_dinosaur.yaml
Active Nav2 config	https://github.com/arassal/carla-nav2-avl/blob/main/ros2_ws/src/perception_costmap/config/real_nav2_params.yaml
Nav2 launch and safety gate	https://github.com/arassal/carla-nav2-avl/blob/main/ros2_ws/src/perception_costmap/deploy/real_nav2_launch.py
Operations guide	https://github.com/arassal/carla-nav2-avl/blob/main/ros2_ws/src/perception_costmap/deploy/README.md
Contribution workflow	https://github.com/arassal/carla-nav2-avl/blob/main/CONTRIBUTION_GUIDE.md
Status/phase ledger	https://github.com/arassal/carla-nav2-avl/blob/main/FINALIZE.md
Required vehicle stack	https://github.com/Paarseus/IGVC_ROS2
Vehicle operating notes	https://github.com/Paarseus/IGVC_ROS2/blob/main/CLAUDE.md
WebUI service	https://github.com/Paarseus/IGVC_ROS2/blob/main/scripts/systemd/avros-webui.service
Vehicle preflight	https://github.com/Paarseus/IGVC_ROS2/blob/main/scripts/preflight_check.sh

Review note

Claims about completed behavior are phrased as repository evidence, not fresh vehicle verification. Where source and documentation conflict, this document identifies the conflict and recommends a confirming test.

End of handoff